PRADITA University

IF140303-Web Application Development

Session-13:

WebSocket and Publish-Subscribe 2

PRU/SPMI/FR-BM-18/0222

Alfa Yohannis



Contents

PRADITA University

1 Introduction
2 Publish-Subscribe Broker
3 Display Control Module
4 Display-Only Module
5 Summary

Introduction

PRADITA University

- **Context.** In the previous session, WebSocket and the Publish–Subscribe communication model were introduced. Phoenix provides `Phoenix.PubSub` for event broadcasting and `LiveView` for building real-time user interfaces.
- **Problem Statement.** A digital queue system requires multiple clients to observe the same queue number consistently for each service queue, including control panels and passive displays.
- **Main Objective.** This session focuses on using a dedicated `GenServer` *per queue* as a single source of truth, with each queue mapped to its own PubSub topic.
- **Core Components.** The system consists of per-queue Counter GenServers, PubSub channels, a Control LiveView, and a Display-Only LiveView module.
- **Learning Outcomes.** To understand real-time multi-client synchronization using per-queue GenServers and the GenServer–PubSub–LiveView pipeline.

Queue Counter: Initialization

PRADITA University

```elixir
defmodule Hiwi.QueueCounter do
  use GenServer alias Phoenix.PubSub

  # Start the GenServer with an empty map
  def start_link(_) do
    start_link_initial_map_state = %{}
    IO.puts("## start_link_initial_map_state")
    IO.inspect(start_link_initial_map_state)
    GenServer.start_link(__MODULE__, start_link_initial_map_state, name:
        __MODULE__)
  end

  # Initialize with an empty map to store queue numbers
  def init(initial_map_state) do
    {:ok, initial_map_state}
  end
  ...
```

This step initializes the Queue Counter process with an empty in-memory state.

Queue Counter: Update & Broadcast Changes

```elixir
# Public API to set the current number for a specific queue_id
def set_current_number(queue_id, new_number) do
  GenServer.cast(__MODULE__, {:set_current_number, queue_id, new_number})
end

# Handle setting the current number for a queue_id and broadcasting the update
def handle_cast({:set_current_number, queue_id, new_number}, current_map_state) do
  updated_map_state = Map.put(current_map_state, queue_id, new_number)

  PubSub.broadcast(Hiwi.PubSub, "queue_counter:updates:#{queue_id}", {:number_update, queue_id, new_number})

  {:noreply, updated_map_state}
end
```

This updates the internal GenServer state asynchronously using `cast`. Each update is immediately broadcasted to all clients subscribed to the queue topic.

Queue Counter: Responding to Requests

```elixir
...
# Public API to get the current number for a specific queue_id
def get_current_number(queue_id) do
  GenServer.call(__MODULE__, {:get_current_number, queue_id})
end

# Handle getting the current number for a queue_id
def handle_call({:get_current_number, queue_id}, _from, current_map_state) do

  # Default to 0 if queue_id is not found
  current_number = Map.get(current_map_state, queue_id, 0)
  {:reply, current_number, current_map_state}
end
...
```

- This function provides synchronous access to the queue counter.
- The caller waits until the GenServer returns the current value.
- The internal state remains unchanged after the request.

Display Control Module

PRADITA University

```elixir
defmodule HiwiWeb.ModifiableLive do
  use HiwiWeb, :live_view
  alias Hiwi.QueueCounter
  alias Hiwi.Queue
  alias Hiwi.User
  alias Hiwi.Repo
  ...
end
```

- This LiveView module acts as the main control interface.
- It allows authorized users to modify the queue state.
- Database access is handled through the Repo and schemas.
- It communicates with the QueueCounter for real-time updates.

Control Module: Initialization

PRADITA University

```elixir
def mount(params, session, socket) do
  %{"id" => queue_id} = params
  case session do
    %{"user_id" => user_id} ->
      # Subscribe to updates from the QueueCounter
      :ok = Phoenix.PubSub.subscribe(Hiwi.PubSub, "queue_counter:updates:#{
          queue_id}")
      # Get the current number from counter, queue and user from database
      current_number = QueueCounter.get_current_number(queue_id)
      queue = Repo.get!(Queue, queue_id)
      user = Repo.get(User, user_id)
      # set values to socket
      socket = socket
        |> assign(:current_number, current_number)
        |> assign(:queue_id, queue_id)
        |> assign(:user_id, user_id)
        |> assign(:user, user)
        |> assign(:queue, queue)
      # return the socket with layout for live
      {:ok, socket, layout: {HiwiWeb.Layouts, :live}}
    _ ->
      ...
```

Control Module: Initialization

PRADITA University

```
  ...
      {:ok, socket, layout: {HiwiWeb.Layouts, :live}}
    _ ->
      socket =
        socket
        |> put_flash(:error, "You haven't signed in.")
      {:ok, push_navigate(socket, to: "/")}
  end
end
```

- This logic separates authenticated and unauthenticated users.
- Authenticated users proceed to the LiveView with the proper layout.
- Unauthorized access is blocked with a flash error message.
- The user is redirected back to the main entry point securely.

Control Module: Update Database & Counter

```elixir
def handle_event("increment_number", _params, socket) do
  # increment the counter in database
  queue_id = socket.assigns.queue_id
  old_queue = Repo.get!(Queue, socket.assigns.queue_id)
  new_state = %{"current_number" => old_queue.current_number + 1}
  changeset = Queue.changeset(old_queue, new_state)
  Repo.update(changeset)
  # get the current number again from the database
  current_queue = Repo.get!(Queue, socket.assigns.queue_id)
  new_number = current_queue.current_number
  # Update QueueCounter with the new number
  QueueCounter.set_current_number(queue_id, new_number)
  # Update the socket state with the new number
  socket = socket
    |> assign(:current_number, new_number)
    |> assign(:queue_id, queue_id)
  # return socket
  {:noreply,socket}
end
```

Control Module: Handling Updates

PRADITA University

```elixir
def handle_info({:number_update, queue_id, new_number}, socket) do
  # Update the socket state when a new number is broadcasted
  socket =
    socket
    |> assign(:current_number, new_number)
    |> assign(:queue_id, queue_id)
  {:noreply,socket}
end
```

- This callback reacts to updates broadcasted by the QueueCounter.
- It ensures the control interface stays synchronized in real time.
- The socket assigns are updated without reloading the page.
- The UI immediately reflects the latest queue number.

Display-Only Module

PRADITA University

```elixir
defmodule HiwiWeb.DisplayOnlyLive do
  use HiwiWeb, :live_view
  alias Hiwi.QueueCounter
  alias Hiwi.Queue
  alias Hiwi.Repo
  ...
end
```

- This LiveView module is dedicated to passive real-time display.
- It subscribes to PubSub updates without modifying server state.
- Queue data is fetched from the database for initial rendering.
- All updates are driven by broadcasted events from the counter.

Display-Only Module: Initialization

PRADITA University

```elixir
def mount(params, _session, socket) do
  #get queue)_id
  %{"id" => queue_id} = params
  # Subscribe to updates from the QueueCounter
  :ok = Phoenix.PubSub.subscribe(Hiwi.PubSub, "queue_counter:updates:#{queue_id}
    ")
  # Get the initial current number of cases handled from QueueCounter
  current_number = QueueCounter.get_current_number(queue_id)
# get queue from database
  queue = Repo.get!(Queue, queue_id)
# add values to socket
  socket = socket
    |> assign(:current_number, current_number)
    |> assign(:queue_name, queue.name)
    |> assign(:queue_prefix, queue.prefix)
    |> assign(:queue_id, queue_id)
# the return response
  {:ok, socket, layout: false}
end
```

Display-Only Module: Handling Updates

```elixir
  ...
  def handle_info({:number_update, queue_id, new_number}, socket) do
    # Update the socket state when a new number is broadcasted
    socket =
      socket
      |> assign(:current_number, new_number)
      |> assign(:queue_id, queue_id)
    {:noreply,socket}
  end
end
```

- This callback is triggered when a PubSub message is received.
- The incoming pattern must match the broadcasted event structure.
- The socket state is updated using `assign/3`.
- `:noreply` triggers a real-time UI re-render without reload.

Summary

PRADITA University

- **Architecture Overview.** The system applies a per-queue `GenServer` as a counter, combined with topic-based `Phoenix.PubSub` for real-time event broadcasting.
- **Real-Time Synchronization.** Control and display clients are synchronized through PubSub topics, ensuring consistent queue numbers across connected clients.
- **Separation of Concerns.** The Control LiveView is responsible for updating database and counters, while the Display-Only LiveView reacts passively to broadcasted updates.
- **Scalability and Isolation.** Using per-queue processes and PubSub communication improves fault isolation and supports scalable multi-queue systems.
- **Key Takeaway.** Combining `GenServer`, `Phoenix.PubSub`, and `LiveView` enables real-time, multi-client web applications.